

Prof. Dr. Stefan Leutenegger

Teaching Assistants: Dr. David Rohr and Anran Zhang

Exercise 3: Wheeled Robot Model

Session date: 17/03/2026

In this exercise, you learn about the kinematics of wheeled mobile bases. You will derive the kinematic model of a Swedish wheel, use it to model an omnidirectional mobile base, and combine it with the manipulator kinematics of the Tiago robot to perform whole-body inverse kinematics for tracking of an end-effector position trajectory.

Task 1: The 90 Degree Swedish Wheel

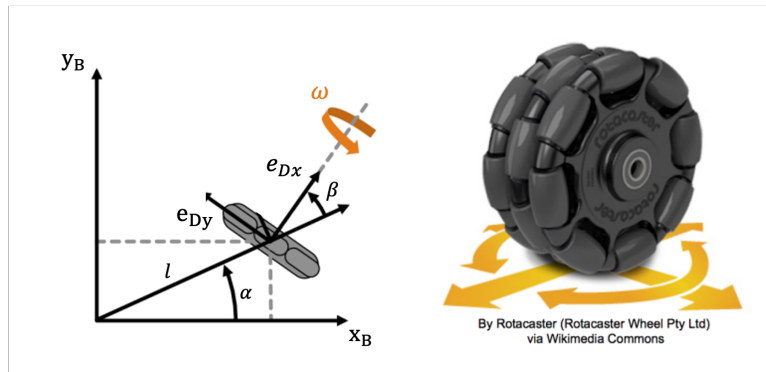


Figure 1: Kinematic parameterization in base frame and physical design of a 90 degree Swedish wheel.

First, we consider a single 90 degree Swedish wheel and derive its constraint equations. Consider the problem parametrization shown in Fig. 1. For convenience, we adopt polar coordinates for the wheel location in the robot base frame $\mathcal{F}_B$, i.e. the angle α and distance l to the wheel center, and the relative wheel orientation β . The radius of the big wheel is r . Recall that the Swedish wheel has one large wheel, and many small freely-spinning wheels around its circumference. The only actuator available is the one that spins the large wheel (primary rotation axis). Movement parallel to the primary wheel rotation axis is free due to the rolling of the small wheels. Derive the rolling constraints c consistent with the form

$$c \cdot {}_B\nu_{WB} = \omega$$

where ${}_B\nu_{WB} := [{}_B\nu_{WB,x}, {}_B\nu_{WB,y}, \Omega_{WB}]^T$ is the twist of the robot's base expressed in the robot frame, and ω is the rotational speed (radians per second) of the wheel around its primary axis. c is a 1×3 matrix.

1. How many degrees of freedom does this wheel have?

Three, 1) rotation around primary axis, 2) free rotation around small rollers, 3) free rotation around contact point.

2. Write out c in terms of α , β , l , r . Pay attention to the definition of positive spinning direction of the wheel.

$$\mathbf{c} = [\sin(\alpha + \beta)/r, -\cos(\alpha + \beta)/r, -\cos(\beta)l/r]$$

3. (Python) Implement the corresponding function `get_wheel_constraint_jacobian` in the Python notebook provided with this exercise.

See `kinematics_tutorial_ex3_sol.ipynb`

Task 2: Omnidirectional Robot Kinematics

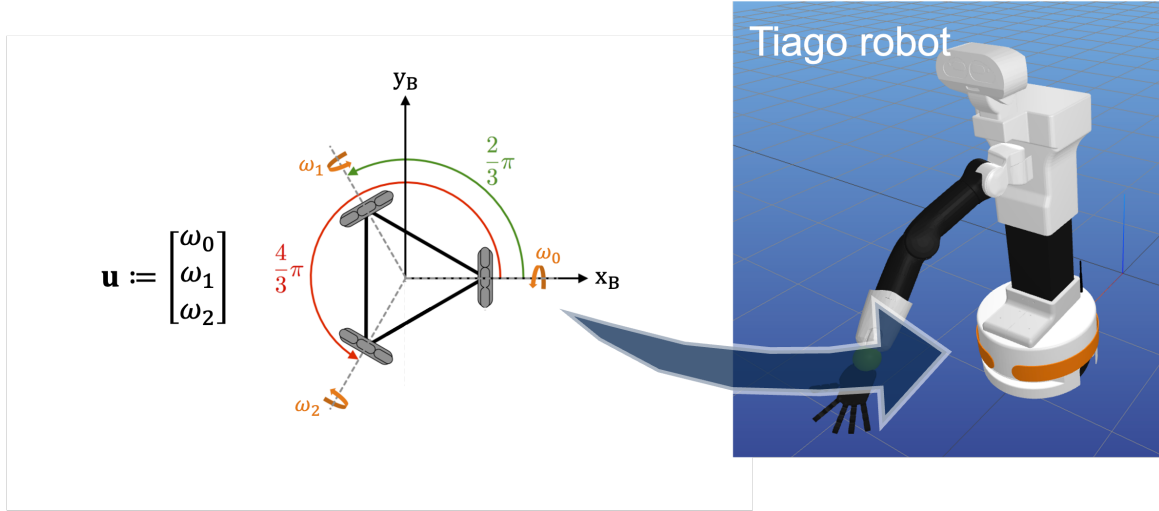


Figure 2: Layout of an omnidirectional mobile base for ground robots using three 90 degree Swedish wheels.

Now, we will derive the stacked wheel equation for the omnidirectional robot base shown in Fig. 2, again considering 90 degree Swedish wheels. These equations then serve to simulate, and move, the Tiago robot introduced in Exercise 2. To complete the ground work for whole-body inverse kinematics in Task 3, you derive the position Jacobian of the end-effector.

Note that Pinocchio, the kinematics library used Exercise 2, can only return kinematic quantities w.r.t base, but not the world once we consider a floating base where $\mathcal{F}_W \neq \mathcal{F}_B$. Pinocchio is not aware of wheel-related motion constraints (not the ones in the URDF, let alone the custom ones considered here), it assumes they spin in free air and don't affect the kinematics. Therefore, the wheel-related components need to be coded-up from scratch, i.e. without pinocchio API.

1. (Python) Define the individual wheel parameters α_i, β_i for the three wheels according to shown arrangement and the definitions in Fig. 1, and enter the values in the dict provided in the code. Use $r_i = 0.1\text{m}$, $l_i = 0.25\text{m}$.

See `kinematics_tutorial_ex3_sol.ipynb`

2. (Python) Derive the Jacobian $\mathbf{J}_B$ which relates the wheel speeds $\mathbf{u} = [\omega_0, \omega_1, \omega_2]^T$ and the twist of the base ${}^B\boldsymbol{\nu}_{WB} := [{}^B\mathbf{v}_{WB,x}, {}^B\mathbf{v}_{WB,y}, \Omega_{WB}]^T$ as

$${}^B\boldsymbol{\nu}_{WB} = \mathbf{J}_B \mathbf{u}$$

and implement it in the function `get_floating_base_jacobian`.

$$\mathbf{J}_B = \begin{bmatrix} \sin(\alpha_0 + \beta_0)/r_0 & -\cos(\alpha_0 + \beta_0)/r_0 & -\cos(\beta_0)l_0/r_0 \\ \sin(\alpha_1 + \beta_1)/r_1 & -\cos(\alpha_1 + \beta_1)/r_1 & -\cos(\beta_1)l_1/r_1 \\ \sin(\alpha_2 + \beta_2)/r_2 & -\cos(\alpha_2 + \beta_2)/r_2 & -\cos(\beta_2)l_2/r_2 \end{bmatrix}^{-1}$$

3. (Python) Implement `step_base` which propagates the base pose over a small time step for a given set of wheel speeds. This provides you with the basic functionality to move Tiago around. Can you pick a set of wheel speeds to have it move forward? What wheel speeds should be picked so that it turns on the spot?

See `kinematics_tutorial_ex3_sol.ipynb`. To obtain a desired base-twist ${}_B\mathbf{v}_{WB}^{ref}$, we simply pick $\mathbf{u} = \mathbf{J}_B^{-1} {}_B\mathbf{v}_{WB}^{ref}$. Note that $\mathbf{J}_B$ is not generally invertible. E.g. a differential-drive base cannot move sideways.

4. (Advanced, Python) Finally, implement the functionality to calculate the end-effector (E) position Jacobian $\mathbf{J}_E(\boldsymbol{\theta})$ with respect to both the joints of Tiago and the wheels of the base, expressed in base-frame, i.e.

$${}_B\mathbf{v}_{WE} = \mathbf{J}_E(\boldsymbol{\theta}) \begin{bmatrix} \dot{\boldsymbol{\theta}} \\ \mathbf{u} \end{bmatrix}$$

Hint, first determine the matrix $\mathbf{A}(\boldsymbol{\theta})$ which, assuming a ‘frozen’ arm $\dot{\boldsymbol{\theta}} = 0$, relates the base-twist and end-effector velocity as ${}_B\mathbf{v}_{WE} = \mathbf{A}(\boldsymbol{\theta}) {}_B\mathbf{v}_{WB}$. With the result from Task 2.2, you can then use $\mathbf{A}(\boldsymbol{\theta})\mathbf{J}_B$ (to map from wheel speeds to ${}_B\mathbf{v}_{WE}$) and concatenate it with the (base-relative) position Jacobian of the end-effector with respect to joints $\boldsymbol{\theta}$ (from Pinocchio) to obtain $\mathbf{J}_E(\boldsymbol{\theta})$

$$\mathbf{A}(\boldsymbol{\theta}) = \begin{bmatrix} 1 & 0 & -{}_B\mathbf{t}_{BE,y}(\boldsymbol{\theta}) \\ 0 & 1 & {}_B\mathbf{t}_{BE,x}(\boldsymbol{\theta}) \\ 0 & 0 & 0 \end{bmatrix}$$

$$\mathbf{J}_E(\boldsymbol{\theta}) = [\mathbf{J}_{E,joints}(\boldsymbol{\theta}), \mathbf{A}(\boldsymbol{\theta})\mathbf{J}_B]$$

Task 3: Inverse Kinematic for Whole-Body Reach (Advanced)

Similar to Exercise 2, we will finally apply an iterative inverse kinematics approach to track a moving target with the end-effector. This time, however, we have a moving base and can therefore also approach targets that are initially out of reach.

1. (Python) Implement the missing parts of `solve_numerical_ik_pos`

See `kinematics_tutorial_ex3_sol.ipynb`

2. (Python) We provided two approaches to obtain the joint and wheel velocities, once using the vanilla pseudo-inverse, and once with a weighted pseudo-inverse that allows to trade-off the individual efforts of joints and wheels. Feel free to explore the effect of different weights (the larger the weight, the more the corresponding joint/wheel is restricted).